CSE 317: Algorithms — Design and Analysis

Project Topic Reference Guide

Comprehensive list of algorithm topics suitable for 5-member groups

PART 1: Detailed Easy Topics (with Complexity)

1. Sorting

#	Algorithm	Design Technique	Complexity
1	Bubble Sort	Brute Force	O(n<super>2</super>)
2	Insertion Sort	Incremental	O(n<super>2</super>)
3	Merge Sort	Divide & Conquer	O(n log n)
4	Quick Sort	Divide & Conquer	O(n log n) avg
5	Heap Sort	Heap Data Structure	O(n log n)

2. Searching

#	Algorithm	Design Technique	Complexity
1	Linear Search	Brute Force	O(n)
2	Binary Search	Divide & Conquer	O(log n)
3	Jump Search	Block-based	O(n<super>0.5</super>)
4	Interpolation Search	Estimation-based	O(log log n) avg
5	Exponential Search	Doubling + Binary	O(log n)

3. Fibonacci Number

#	Algorithm	Design Technique	Complexity
1	Naive Recursion	Recursion	O(2<super>n</super>)
2	Top-down DP (Memoization)	Dynamic Programming	O(n)
3	Bottom-up DP	Dynamic Programming	O(n)
4	Matrix Exponentiation	Divide & Conquer	O(log n)
5	Binet's Formula	Mathematical / Closed-form	O(1)

4. 0/1 Knapsack

#	Algorithm	Design Technique	Complexity
1	Brute Force	Exhaustive Search	O(2<super>n</super>)
2	Recursion	Recursion	O(2<super>n</super>)
3	Top-down DP (Memoization)	Dynamic Programming	O(nW)
4	Bottom-up DP	Dynamic Programming	O(nW)
5	Branch & Bound	Pruning	Better than O(2<super>n</super>)

5. Coin Change (Minimum Coins)

#	Algorithm	Design Technique	Complexity
1	Greedy	Greedy	Not always optimal
2	Naive Recursion	Recursion	O(S<super>n</super>)
3	Top-down DP (Memoization)	Dynamic Programming	O(n·S)
4	Bottom-up DP	Dynamic Programming	O(n·S)
5	BFS	Graph Traversal	O(n·S)

6. String Matching

#	Algorithm	Design Technique	Complexity
1	Brute Force	Exhaustive Search	O(n·m)
2	KMP	Preprocessing	O(n+m)
3	Rabin-Karp	Hashing	O(n+m) avg
4	Boyer-Moore	Heuristics	O(n/m) best
5	Z-Algorithm	Preprocessing	O(n+m)

7. Longest Common Subsequence (LCS)

#	Algorithm	Design Technique	Complexity
1	Naive Recursion	Recursion	O(2<super>n</super>)
2	Top-down DP	Dynamic Programming	O(n·m)
3	Bottom-up DP	Dynamic Programming	O(n·m)
4	Hirschberg's	Divide & Conquer + DP	O(n·m) time, O(n) space
5	Hunt-Szymanski	Hashing + DP	O(r log n)

8. Greatest Common Divisor (GCD)

#	Algorithm	Design Technique	Complexity
1	Brute Force	Exhaustive Search	O(min(a,b))
2	Euclidean Algorithm	Recursion	O(log min(a,b))
3	Extended Euclidean	Recursion	O(log min(a,b))
4	Binary / Stein's	Bit Manipulation	O(log<super>2</super> n)
5	Lehmer's	Optimization	O(log<super>2</super> n)

9. Single Source Shortest Path

#	Algorithm	Design Technique	Complexity
1	BFS (unweighted)	Graph Traversal	O(V+E)
2	DFS (unweighted)	Graph Traversal	O(V+E)
3	Dijkstra's	Greedy	O((V+E) log V)
4	Bellman-Ford	Dynamic Programming	O(V·E)
5	A* Search	Heuristic	O(E log V)

10. Minimum Spanning Tree

#	Algorithm	Design Technique	Complexity
1	Brute Force	Exhaustive Search	O(E<super>n</super>)
2	Prim's (simple)	Greedy	O(V<super>2</super>)
3	Prim's (priority queue)	Greedy + Heap	O(E log V)
4	Kruskal's	Greedy + Union-Find	O(E log E)
5	Boruvka's	Greedy	O(E log V)

11. Subset Sum

#	Algorithm	Design Technique	Complexity
1	Brute Force	Exhaustive Search	O(2<super>n</super>)
2	Backtracking	Pruning	O(2<super>n</super>) better
3	Top-down DP	Dynamic Programming	O(n·S)
4	Bottom-up DP	Dynamic Programming	O(n·S)
5	Meet in the Middle	Divide & Conquer	O(2<super>n/2</super>)

12. Primality Testing

#	Algorithm	Design Technique	Complexity
1	Trial Division	Brute Force	O(n<super>0.5</super>)
2	Sieve of Eratosthenes	Elimination	O(n log log n)
3	Fermat's Test	Probabilistic	O(k log<super>2</super> n)
4	Miller-Rabin	Probabilistic	O(k log<super>2</super> n)
5	Solovay-Strassen	Probabilistic	O(k log<super>2</super> n)

13. Matrix Multiplication

#	Algorithm	Design Technique	Complexity
1	Naive / Schoolbook	Brute Force	O(n<super>3</super>)
2	Divide & Conquer	Divide & Conquer	O(n<super>3</super>)
3	Strassen's	Divide & Conquer	O(n<super>2.81</super>)
4	Winograd's	Optimization	O(n<super>3</super>) fewer mults
5	Cache-Oblivious	Memory Optimization	O(n<super>3</super>) better cache

14. Text Compression

#	Algorithm	Design Technique	Complexity
1	Run-Length Encoding	Simple encoding	O(n)
2	Huffman Coding	Greedy	O(n log n)
3	LZ77	Sliding Window	O(n)
4	LZ78	Dictionary-based	O(n)
5	LZW	Dictionary-based	O(n)

15. Maze / Puzzle Solving (8-Puzzle)

#	Algorithm	Design Technique	Complexity
1	BFS	Graph Traversal	O(b<super>d</super>)
2	DFS	Graph Traversal	O(b<super>m</super>)
3	Greedy Best-First	Heuristic	O(b<super>m</super>)
4	A* Search	Heuristic + Optimal	O(b<super>d</super>)
5	IDA*	Iterative Deepening	O(b<super>d</super>) less memory

16. Numerical Integration

#	Algorithm	Design Technique	Complexity
1	Riemann Sum	Approximation	O(n)
2	Trapezoidal Rule	Approximation	O(n)
3	Simpson's 1/3 Rule	Approximation	O(n)
4	Simpson's 3/8 Rule	Approximation	O(n)
5	Monte Carlo Integration	Randomized	O(n)

17. Cache Replacement

#	Algorithm	Design Technique	Complexity
1	FIFO	Queue-based	O(1)

#	Algorithm	Design Technique	Complexity
2	LRU	Stack / HashMap	O(1)
3	LFU	Frequency Count	O(1)
4	Optimal (Belady's)	Offline / Lookahead	O(n log n)
5	Clock Algorithm	Circular Buffer	O(1)

18. Convex Hull

#	Algorithm	Design Technique	Complexity
1	Brute Force	Exhaustive Search	O(n<super>3</super>)
2	Jarvis March	Greedy	O(n·h)
3	Graham Scan	Sorting + Stack	O(n log n)
4	Quickhull	Divide & Conquer	O(n log n) avg
5	Chan's Algorithm	Optimal	O(n log h)

19. Maximum Subarray

#	Algorithm	Design Technique	Complexity
1	Brute Force (all pairs)	Exhaustive Search	O(n<super>3</super>)
2	Better Brute Force	Exhaustive Search	O(n<super>2</super>)
3	Divide & Conquer	Divide & Conquer	O(n log n)
4	Kadane's Algorithm	Dynamic Programming	O(n)
5	Prefix Sum	Preprocessing	O(n)

Top Recommended Topics from Part 1

Topic	Why It Is Great
Sorting	Everyone knows it, easy to implement and benchmark
Fibonacci	Perfectly shows recursion to DP to math progression
Coin Change	Great DP story, easy to visualize
String Matching	Practical, interesting algorithms, clear differences
Shortest Path	Visual graphs, very presentable
Max Subarray	Huge complexity difference between algorithms
Primality Testing	Math-heavy but easy to code
Numerical Integration	Easy to implement, great graphs

PART 2: Fresh Topics (Not in Standard Lists)

1. Game Theory / Decision

#	Algorithm	Design Technique
1	Minimax	Recursion
2	Alpha-Beta Pruning	Pruning
3	Monte Carlo Tree Search	Randomized
4	Negamax	Recursion
5	Iterative Deepening Minimax	Iterative

2. Shuffling / Randomization

#	Algorithm	Design Technique
1	Naive Random Swap	Brute Force
2	Fisher-Yates Shuffle	Randomized
3	Sattolo's Algorithm	Randomized
4	Reservoir Sampling	Streaming
5	Sort-based Shuffle	Sorting

3. Exponentiation (Power of a Number)

#	Algorithm	Design Technique
1	Naive Iterative	Brute Force
2	Naive Recursive	Recursion
3	Fast Power (Binary Exponentiation)	Divide & Conquer
4	Matrix Exponentiation	Divide & Conquer
5	Exponentiation by Squaring (Iterative)	Bit Manipulation

4. Tower of Hanoi

#	Algorithm	Design Technique
1	Classic Recursion	Recursion
2	Iterative (Stack-based)	Stack
3	Frame-Stewart (4 pegs)	Dynamic Programming
4	Binary Representation Method	Bit Manipulation
5	BFS-based	Graph Traversal

5. Bin Packing

#	Algorithm	Design Technique
1	Brute Force	Exhaustive Search
2	First Fit	Greedy
3	Best Fit	Greedy
4	First Fit Decreasing	Greedy + Sorting
5	Best Fit Decreasing	Greedy + Sorting

6. Expression Evaluation / Parenthesis

#	Algorithm	Design Technique
1	Brute Force	Exhaustive Search
2	Stack-based Evaluation	Stack
3	Recursive Descent Parsing	Recursion
4	Shunting-Yard	Stack
5	DP (Matrix Chain style)	Dynamic Programming

7. House Robber / Max Non-Adjacent Sum

#	Algorithm	Design Technique
1	Brute Force	Exhaustive Search
2	Recursion	Recursion
3	Memoization	Dynamic Programming
4	Bottom-up DP	Dynamic Programming
5	Space-optimized DP	Dynamic Programming

8. Counting Paths in a Grid

#	Algorithm	Design Technique
1	Brute Force / DFS	Exhaustive Search
2	BFS	Graph Traversal
3	Recursion	Recursion
4	Top-down DP	Dynamic Programming
5	Bottom-up DP	Dynamic Programming

9. Anagram Detection

#	Algorithm	Design Technique
1	Sorting-based	Sorting
2	Frequency Count (Array)	Counting
3	HashMap-based	Hashing
4	XOR / Prime Product	Bit / Math
5	Sliding Window (all anagrams)	Two Pointers

10. Flood Fill / Connected Components

#	Algorithm	Design Technique
1	Recursive DFS	Recursion
2	Iterative DFS (Stack)	Stack
3	BFS	Queue
4	Union-Find	Disjoint Set
5	Scan-line Fill	Scan Line

11. CPU / Task Scheduling

#	Algorithm	Design Technique
1	FCFS (First Come First Serve)	Queue
2	SJF (Shortest Job First)	Greedy
3	Round Robin	Cyclic
4	Priority Scheduling	Greedy + Heap
5	Earliest Deadline First	Greedy

12. Cycle Detection in a Graph

#	Algorithm	Design Technique
1	DFS with visited array	Graph Traversal
2	BFS (Kahn's Topological)	Graph Traversal
3	Union-Find	Disjoint Set
4	Floyd's Tortoise & Hare	Two Pointers
5	Coloring Method	Graph Coloring

13. Number Guessing / Binary Search Variants

#	Algorithm	Design Technique
1	Linear Scan	Brute Force
2	Binary Search	Divide & Conquer
3	Ternary Search	Divide & Conquer
4	Fibonacci Search	Fibonacci-based
5	Exponential + Binary	Doubling

14. Water Jug Problem

#	Algorithm	Design Technique
1	Brute Force	Exhaustive Search
2	BFS	Graph Traversal
3	DFS	Graph Traversal
4	GCD-based Mathematical	Mathematical
5	A* Search	Heuristic

15. Pascal's Triangle / Binomial Coefficients

#	Algorithm	Design Technique
1	Naive Recursion	Recursion
2	Memoization	Dynamic Programming
3	Bottom-up DP (Triangle)	Dynamic Programming
4	Multiplicative Formula	Mathematical
5	Pascal's Row Iteration	Space Optimization

16. Edit Distance (Levenshtein)

#	Algorithm	Design Technique
1	Naive Recursion	Recursion
2	Memoization	Dynamic Programming
3	Bottom-up DP	Dynamic Programming
4	Hirschberg's (space opt.)	Divide & Conquer
5	Ukkonen's Threshold	Optimization

17. Random Number Generation

#	Algorithm	Design Technique
1	Linear Congruential Generator	Mathematical

#	Algorithm	Design Technique
2	Mersenne Twister	Bit Manipulation
3	Xorshift	Bit Manipulation
4	LFSR (Linear Feedback Shift Register)	Bit Manipulation
5	Middle Square Method	Mathematical

18. Coin Change — Count All Ways

#	Algorithm	Design Technique
1	Naive Recursion	Recursion
2	Memoization	Dynamic Programming
3	Bottom-up DP	Dynamic Programming
4	BFS	Graph Traversal
5	Generating Functions	Mathematical

Top Recommended Topics from Part 2

Topic	Why It Is Great
Exponentiation	Clean, huge complexity jump, easy to code
Bin Packing	Great greedy story, real-world relevance
CPU Scheduling	OS relevance, easy to simulate and benchmark
Edit Distance	Very practical, NLP relevance, clear DP progression
Cycle Detection	Short code, multiple clever techniques
Flood Fill	Visual, fun to demo, easy to implement
Anagram Detection	5 surprisingly different approaches to a tiny problem
Pascal's Triangle	Simple but shows recursion to DP to math beautifully